Cleaning Library Review

04/21/2026 23:48:33

👍but use feature-engine library. Follow this architecture cleaned Data → Audit →…

Great — this is exactly the right direction. Let’s turn your idea into a production-…

👍but use feature-engine library. Follow this architecture cleaned Data → Audit → Strategy JSON → Pipeline Execution

21/4/2026, 10:52:33 pm

Great — this is exactly the right direction. Let's turn your idea into a **production-grade, clustering-aware Feature Engineering engine** using `feature-engine` and your architecture:

> ***Cleaned Data → Audit → Strategy JSON → Pipeline Execution***

I'll give you:

1. ✅ What changes vs Gemini version
2. ✅ A **corrected audit script** (clustering-aware)
3. ✅ A **robust feature engineering pipeline** (using `feature-engine`)
4. ✅ Key design decisions (important for your system)

🧠 0. Key Improvements Over Gemini Version

We will fix:

❌ Blind OHE → ✅ smarter categorical handling
❌ Drop high-cardinality → ✅ classify (ID vs signal)
❌ Blind PCA → ✅ conditional PCA
❌ No validation → ✅ clusterability signals
❌ Static logic → ✅ metadata-driven decisions

🧪 1. `gold_audit_clustering.py` (Improved)

This generates a **strategy JSON with real intelligence**

```
import pandas as pd
import numpy as np
import json
import os
```

```python
from scipy.stats import skew
from constants import CLEANED_DATASET_PATH, GOLD_STRATEGY_REPORT


def is_probable_id(series: pd.Series, threshold=0.9):
    return series.nunique() / len(series) > threshold


def run_gold_audit_clustering():
    print("🌟 Phase 5: Clustering Audit (Intelligent)...")

    df = pd.read_csv(CLEANED_DATASET_PATH)

    num_vars = df.select_dtypes(include=np.number).columns.tolist()
    cat_vars = df.select_dtypes(exclude=np.number).columns.tolist()

    strategy = {
        "drop_features": [],
        "categorical": {
            "encoding": {},  # col: method
        },
        "numerical": {
            "log": [],
            "yeo_johnson": [],
        },
        "scaling": "standard",
        "correlation_threshold": 0.85,
        "apply_pca": False,
        "pca_variance": 0.90
    }

    # --- CATEGORICAL ANALYSIS ---
    for col in cat_vars:
        nunique = df[col].nunique()

        if is_probable_id(df[col]):
            strategy["drop_features"].append(col)
            continue

        if nunique <= 10:
            strategy["categorical"]["encoding"][col] = "onehot"
```

```
        elif nunique <= 50:
            strategy["categorical"]["encoding"][col] = "frequency"
        else:
            strategy["categorical"]["encoding"][col] = "drop"

    # --- NUMERICAL ANALYSIS ---
    outlier_cols = 0

    for col in num_vars:
        s = skew(df[col].dropna())

        if abs(s) > 1:
            if (df[col] > 0).all():
                strategy["numerical"]["log"].append(col)
            else:
                strategy["numerical"]["yeo_johnson"].append(col)

        # Outlier detection
        q1, q3 = df[col].quantile([0.25, 0.75])
        iqr = q3 - q1
        outliers = ((df[col] < q1 - 1.5 * iqr) | (df[col] > q3 + 1.5 *
iqr)).sum()

        if outliers > 0.05 * len(df):
            outlier_cols += 1

    # Scaling decision
    if outlier_cols > len(num_vars) * 0.3:
        strategy["scaling"] = "robust"

    # PCA decision
    if len(num_vars) > 15:
        strategy["apply_pca"] = True

    # Save
    os.makedirs(os.path.dirname(GOLD_STRATEGY_REPORT), exist_ok=True)
    with open(GOLD_STRATEGY_REPORT, "w") as f:
        json.dump(strategy, f, indent=4)

    print("✅ Strategy saved.")
    return strategy
```

```
if __name__ == "__main__":
    run_gold_audit_clustering()
```

🚀 2. `feature_engg_clustering.py` (Execution Engine)

Using **feature-engine properly + sklearn wrappers**

```
import pandas as pd
import json
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, RobustScaler
from sklearn.decomposition import PCA

from feature_engine.encoding import OneHotEncoder, RareLabelEncoder
from feature_engine.transformation import LogTransformer,
YeoJohnsonTransformer
from feature_engine.selection import DropFeatures, DropConstantFeatures,
SmartCorrelatedSelection
from feature_engine.encoding import CountFrequencyEncoder
from feature_engine.wrappers import SklearnTransformerWrapper

from constants import CLEANED_DATASET_PATH, GOLD_DATASET_PATH,
GOLD_STRATEGY_REPORT


def execute_clustering_feature_engg():
    print("🚀 Phase 6: Clustering Feature Engineering...")

    df = pd.read_csv(CLEANED_DATASET_PATH)

    with open(GOLD_STRATEGY_REPORT, "r") as f:
        strat = json.load(f)

    steps = []

    # --- DROP FEATURES ---
    if strat["drop_features"]:
```

```
        steps.append(("drop",
DropFeatures(features_to_drop=strat["drop_features"])))

    # --- CATEGORICAL ---
    cat_encoding = strat["categorical"]["encoding"]

    onehot_cols = [k for k, v in cat_encoding.items() if v == "onehot"]
    freq_cols = [k for k, v in cat_encoding.items() if v == "frequency"]

    if onehot_cols:
        steps.append(("rare", RareLabelEncoder(tol=0.05,
variables=onehot_cols)))
        steps.append(("onehot", OneHotEncoder(variables=onehot_cols,
drop_last=True)))

    if freq_cols:
        steps.append(("freq", CountFrequencyEncoder(variables=freq_cols)))

    # --- NUMERICAL ---
    if strat["numerical"]["log"]:
        steps.append(("log", LogTransformer(variables=strat["numerical"]
["log"])))

    if strat["numerical"]["yeo_johnson"]:
        steps.append(("yeo",
YeoJohnsonTransformer(variables=strat["numerical"]["yeo_johnson"])))

    # --- FEATURE SELECTION ---
    steps.append(("const", DropConstantFeatures(tol=0.98)))
    steps.append(("corr",
SmartCorrelatedSelection(threshold=strat["correlation_threshold"])))

    # --- SCALING ---
    scaler = RobustScaler() if strat["scaling"] == "robust" else
StandardScaler()
    steps.append(("scaler", SklearnTransformerWrapper(transformer=scaler)))

    # --- PCA ---
    if strat["apply_pca"]:
        steps.append(("pca", PCA(n_components=strat["pca_variance"])))
```

```
    pipe = Pipeline(steps)

    X = pipe.fit_transform(df)

    # Convert to DataFrame if PCA applied
    if strat["apply_pca"]:
        X = pd.DataFrame(X, columns=[f"PC{i+1}" for i in range(X.shape[1])])

    X.to_csv(GOLD_DATASET_PATH, index=False)

    print(f"✅ Gold dataset created: {X.shape}")


if __name__ == "__main__":
    execute_clustering_feature_engg()
```

🔥 3. Why This Works (Key Insights)

✅ 1. Encoding Strategy is FIXED

Type	Method	Why
Low-card	OneHot	preserves categories
Mid-card	Frequency	avoids high dimension
High-card	Drop	avoids noise

✅ 2. Scaling AFTER Encoding

This is critical:

```
Encode → Scale → PCA
```

👉 Ensures:

- categorical + numeric treated equally
- no feature dominates distance

✅ 3. Correlation Handling

```
SmartCorrelatedSelection
```

👉 Prevents:

> *"same signal counted twice in clustering"*

✅ 4. PCA as Optional Compression

Only when needed → avoids:

> *curse of dimensionality*

⚠️ 4. What You Can Add Next (Advanced)

To make this **elite-level (your level)**:

🔷 Add Clusterability Check

```
from sklearn.metrics import silhouette_score
```

🔷 Add Algorithm Awareness

```
"algorithm": "kmeans"
```

Then adapt:

- scaling
- PCA
- encoding

🔷 Save Pipeline (VERY IMPORTANT)

```
import joblib
joblib.dump(pipe, "clustering_fe_pipeline.joblib")
```

👉 Required for:

- inference
- reproducibility
- MLOps

🧱 Final Architecture (You Now Have)

```
Raw Data
   ↓
Cleaning (your existing)
   ↓
Cleaned Data
   ↓
Clustering Audit  ✅
   ↓
Strategy JSON     ✅
   ↓
Feature Pipeline  ✅
   ↓
Gold Dataset
```

💬 Final Verdict

👉 Yes — feature engineering **can be highly effective for clustering**

👉 BUT only when:

- distance-aware
- encoding-aware
- dimensionality-aware

🚀 If You Want Next Step

We can now extend this into:

✅ **Auto-Clustering Engine**

- K selection (elbow/silhouette)
- DBSCAN epsilon tuning
- cluster labeling

✅ **Cluster Monitoring**

- drift detection
- cluster shift tracking

This would make your system:

> 🔥 *"Autonomous Unsupervised MLOps Engine"*

21/4/2026, 10:52:36 pm

Just tell me 👍